

ACID Properties in DBMS

A Deep Dive with Detailed Examples, Practical Scenarios & Rationale

Atomicity • Consistency • Isolation • Durability

Table of Contents

6.1 Transaction Concept

6.2 ACID Properties

- 6.2.1 Atomicity — “All or Nothing”
- 6.2.2 Consistency — “Valid State to Valid State”
- 6.2.3 Isolation — “Concurrent Transactions Don't Interfere”
- 6.2.4 Durability — “Once Committed, Always Committed”

6.3 Serializability

- 6.3.1 Simple Example: Serial vs. Non-Serial Schedule
- 6.3.2 Conflict Serializability
- 6.3.3 View Serializability (Brief)

6.4 Concurrency Control

- 6.4.1 Need for Concurrency Control
- 6.4.2 Lock-Based Protocols

6.5 Recovery

- 6.5.1 Failure Classification
- 6.5.2 Shadow Paging (Recovery Technique)

6.6 Putting It All Together: Full Bank Transfer Walkthrough

6.7 Why ACID Matters — Real-World Stakes

- 6.7.1 Where ACID is Critical
- 6.7.2 The Trade-off: ACID vs. BASE

6.8 Summary

6.1 Transaction Concept

A transaction is a sequence of one or more database operations (reads and writes) that is treated as a single logical unit of work. The classic example is a bank transfer: money must be debited from one account and credited to another as one indivisible operation. ACID is the acronym for the four properties — Atomicity, Consistency, Isolation, Durability — that guarantee a transaction is processed reliably, even in the presence of crashes, power failures, or many concurrent users hitting the database at the same time.

Without ACID guarantees, databases would be unpredictable: money could vanish during transfers, two customers could book the same airline seat, or a confirmed order could disappear after a server crash. ACID is what allows applications like banking, e-commerce, airline reservation, and healthcare systems to be trusted with critical data.

Why this matters

Every relational database you have used — MySQL, PostgreSQL, Oracle, SQL Server — is built around ACID guarantees at its core. Understanding ACID deeply is essential for designing correct schemas, debugging concurrency bugs, and making informed decisions when choosing between SQL and NoSQL systems.

6.2 ACID Properties

ACID is the acronym for the four properties — Atomicity, Consistency, Isolation, and Durability — that together guarantee a database transaction is processed reliably. Each is explored below with detailed, real-world examples.

6.2.1 Atomicity — “All or Nothing”

Atomicity guarantees that a transaction is treated as a single, indivisible unit. Either every operation within the transaction is completed successfully, or none of them are applied at all. There is no partial execution.

Example 1: Bank Transfer

Transferring ₹5000 from Account A to Account B requires two operations:

```
BEGIN TRANSACTION;  
  UPDATE Account SET balance = balance - 5000 WHERE acc_id = 'A';  
  UPDATE Account SET balance = balance + 5000 WHERE acc_id = 'B';  
COMMIT;
```

If the system crashes right after the first UPDATE but before the second, atomicity ensures that on recovery, the DBMS rolls back the first UPDATE as well. Account A is restored to its original balance — the money is never “lost in transit.”

Example 2: Order + Stock Deduction (Simple Two-Step Case)

Placing an order needs just two things to succeed together: create the order, and reduce stock by one unit.

```
BEGIN TRANSACTION;  
  INSERT INTO Orders (order_id, product_id) VALUES (5001, 101);  
  UPDATE Inventory SET stock = stock - 1 WHERE product_id = 101;  
COMMIT;
```

Suppose stock for product 101 is already 0 when the UPDATE runs, so it fails. Atomicity ensures the INSERT into Orders is rolled back too — you never end up with an order row for a product that was never actually reserved.

Example 3: Airline Seat Booking

A booking transaction (1) charges the customer's card, and (2) marks seat 14C as reserved. If the payment gateway call succeeds but the seat-reservation UPDATE fails (e.g., due to a deadlock), atomicity requires reversing the charge — otherwise the customer pays for a seat they never received.

How Atomicity is Implemented

- Write-Ahead Logging (WAL): every change is written to a log before being applied to the actual data files, so incomplete transactions can be undone on recovery.
- Shadow Paging: an alternative technique where changes are made to a copy of data pages, and the pointer to the “current” version is only switched once all changes succeed.
- Rollback segments / undo logs: used by databases like Oracle to reverse partial changes when a transaction aborts.

Practical Scenario

Scenario: Payment gateway timeout

A user pays for a movie ticket. The system deducts the amount and tries to lock the seat, but the seat-locking service times out. Atomicity ensures the payment deduction transaction and seat lock are part of one atomic unit (often

coordinated via a two-phase commit across services) — so the customer is refunded automatically instead of losing money with no ticket.

What Happens Without Atomicity

Failure scenario

If atomicity is not enforced, a crash mid-transfer could leave Account A debited but Account B never credited — money effectively disappears from the system. This kind of bug is catastrophic in financial systems and extremely difficult to trace after the fact.

6.2.2 Consistency — “Valid State to Valid State”

Consistency ensures a transaction can only bring the database from one valid state to another valid state, preserving all defined rules: constraints, data types, foreign keys, triggers, and cascading rules. If a transaction would violate any rule, the database rejects it entirely.

Example 1: Balance Constraint

Suppose the Account table enforces a CHECK constraint that balance can never go negative:

```
ALTER TABLE Account ADD CONSTRAINT chk_balance CHECK (balance >= 0);

BEGIN TRANSACTION;
  -- Account A only has 8000
  UPDATE Account SET balance = balance - 10000 WHERE acc_id = 'A';
COMMIT;
-- Result: ERROR - violates check constraint 'chk_balance'
-- Transaction is rolled back automatically
```

The transaction is rejected outright, and the account balance remains untouched at 8000. Consistency prevented the database from ever holding an invalid value like -2000.

Example 2: Referential Integrity

An OrderItems row must always reference an existing Product:

```
ALTER TABLE OrderItems
  ADD CONSTRAINT fk_product
  FOREIGN KEY (product_id) REFERENCES Products(product_id);

INSERT INTO OrderItems (order_id, product_id, qty) VALUES (5002, 9999, 2);
-- Result: ERROR - product_id 9999 does not exist in Products
```

Without this rule, you could end up with “orphaned” order items pointing to products that don't exist, breaking every report or query that joins the two tables.

Example 3: Business Invariant Across Rows

Imagine a warehouse system where the invariant is: `reserved_stock + available_stock = total_stock`, enforced by a trigger. If a transaction updates `available_stock` without correspondingly updating `reserved_stock`, the trigger raises an error and the transaction is rolled back, keeping the invariant intact at all times.

Practical Scenario

Scenario: Double-entry accounting system

In accounting software, every debit must have a matching credit (the books must always balance). Consistency, enforced through application logic and database constraints together, guarantees that a transaction posting only a debit without its matching credit is rejected — keeping the ledger mathematically correct at every point in time.

Note: Consistency is a shared responsibility. The DBMS enforces the constraints, triggers, and rules — but you, the developer/DBA, must define correct ones. A missing constraint means the DBMS has nothing to enforce, even though the property is technically “active.”

6.2.3 Isolation — “Concurrent Transactions Don't Interfere”

Isolation ensures that the result of running multiple transactions concurrently is the same as if they had been run one after another (serially) — even though, physically, they may overlap in time. This is what prevents race conditions.

Example 1: Double Booking (Lost Update Problem)

Two users try to book the last available seat on a flight at the same time:

Time	Transaction T1 (User X)	Transaction T2 (User Y)
t1	READ seat_status → 'available'	
t2		READ seat_status → 'available'
t3	UPDATE seat_status = 'booked', user = 'X'	
t4	COMMIT	
t5		UPDATE seat_status = 'booked', user = 'Y'
t6		COMMIT

Without isolation, both T1 and T2 read 'available' before either commits, so both believe they succeeded — resulting in a double booking. With proper isolation (row-level locking or MVCC), T2 would be blocked from reading/updating the row until T1 finishes, then see 'booked' and fail or retry.

Example 2: Dirty Read

Step	Transaction T1	Transaction T2 (Read Uncommitted)
1	BEGIN TRANSACTION;	
2	UPDATE Account SET balance = 5000 WHERE acc_id = 'A'; -- not committed yet	
3		BEGIN TRANSACTION;
4		SELECT balance FROM Account WHERE acc_id = 'A'; -- reads 5000 <-- DIRTY!
5	ROLLBACK; -- balance reverts to 8000	
6		-- T2 already used value 5000,
		-- which never really existed

This is called a dirty read — T2 saw a value that was never actually committed. Higher isolation levels (Read Committed and above) prevent this by only allowing reads of committed data.

Example 3: Non-Repeatable Read

T1 reads a row twice within the same transaction, but T2 commits an update to that row in between the two reads — so T1 gets two different values for the same query, which can break reports that assume a value stays fixed during a transaction. The Repeatable Read isolation level prevents this.

Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

Higher isolation levels give stronger correctness guarantees but reduce concurrency (more locking/blocking, more transaction retries), so there is always a trade-off between correctness and throughput.

Detailed Example of Each Isolation Level

Setup for all four examples below: Account A starts with balance = 8000. Two transactions, T1 and T2, run concurrently against this same row.

(a) READ UNCOMMITTED

The weakest level. A transaction can read data that another transaction has changed but not yet committed. This allows dirty reads.

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

Step	Transaction T1	Transaction T2
1	BEGIN TRANSACTION;	
2	UPDATE Account SET balance = 5000 WHERE acc_id = 'A'; -- (not committed yet)	
3		BEGIN TRANSACTION;
4		SELECT balance FROM Account WHERE acc_id = 'A'; -- reads 5000 <-- DIRTY READ
5	ROLLBACK; -- balance goes back to 8000	
6		-- T2 already used the -- fictitious value 5000

T2 read a value (5000) that never actually existed once T1 rolled back. This is the fastest, least safe option — it is rarely used in practice because incorrect data can leak into business decisions.

(b) READ COMMITTED

Prevents dirty reads: a transaction can only see data that has already been committed by other transactions. This is the default isolation level in most databases, including PostgreSQL, Oracle, and SQL Server.

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

Step	Transaction T1	Transaction T2
1	BEGIN TRANSACTION;	
2	UPDATE Account SET balance = 5000 WHERE acc_id = 'A'; -- (not committed yet)	
3		BEGIN TRANSACTION; SELECT balance FROM Account WHERE acc_id = 'A'; -- BLOCKS, waits for T1
4	COMMIT; -- balance is now 5000	
5		-- unblocks, reads 5000 -- (the committed value, not dirty)

T2 no longer sees uncommitted/dirty data — it correctly waited and read the committed value 5000. However, Read Committed only promises that each individual read sees committed data; it does NOT promise the value will stay the same for the rest of T2's transaction. Continuing the same example with a third transaction, T3:

Step	Transaction T2 (continued)	Transaction T3
6	SELECT balance FROM Account WHERE acc_id = 'A'; -- 1st read: 5000	
7		BEGIN TRANSACTION; UPDATE Account SET balance = 7000 WHERE acc_id = 'A'; COMMIT; -- succeeds
8	SELECT balance FROM Account WHERE acc_id = 'A'; -- 2nd read: 7000 <-- DIFFERENT!	
9	COMMIT;	

T2 asked for the exact same row twice in one transaction and got two different answers (5000, then 7000) because T3 slipped in and committed a change in between. This is the non-repeatable read: each read was individually valid and committed, but the value was not stable across the lifetime of T2's transaction. Repeatable Read (next) is specifically designed to prevent this.

(c) REPEATABLE READ

Guarantees that if a transaction reads a row once, it will see the exact same value if it reads that row again, for the entire duration of the transaction — even if another transaction commits a change in the meantime. MySQL's InnoDB engine uses this as its default level.

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

Step	Transaction T1	Transaction T2
1	BEGIN TRANSACTION;	

Step	Transaction T1	Transaction T2
2	SELECT balance FROM Account WHERE acc_id = 'A'; -- reads 8000	
3		BEGIN TRANSACTION; UPDATE Account SET balance = 5000 WHERE acc_id = 'A';
4		COMMIT; -- succeeds
5	SELECT balance FROM Account WHERE acc_id = 'A'; -- STILL reads 8000 (repeatable!) -- T1 keeps a consistent snapshot	
6	COMMIT; -- balance now visible as 5000 -- to new transactions	

T1's two reads returned the identical value (8000), because Repeatable Read isolates T1 from T2's committed change until T1 itself finishes. However, Repeatable Read only guarantees that rows you have already read stay the same — it does NOT block a different transaction from inserting a brand-new row that would match your query's range condition. Continuing the example with a range query and a third transaction, T3:

Step	Transaction T1 (continued)	Transaction T3
7	SELECT COUNT(*) FROM Account WHERE branch = 'Delhi'; -- 1st count: 5 accounts	
8		BEGIN TRANSACTION; INSERT INTO Account (acc_id, branch, balance) VALUES ('Z9', 'Delhi', 2000); COMMIT; -- succeeds
9	SELECT COUNT(*) FROM Account WHERE branch = 'Delhi'; -- 2nd count: 6 <-- PHANTOM ROW!	
10	COMMIT;	

T1 ran the exact same range query twice and got two different counts (5, then 6) because T3 inserted a brand-new row that happened to match T1's WHERE condition. Notice this is different from the earlier non-repeatable read: T1 never touched account Z9 directly, and no existing row it had already read was changed — a new row simply “appeared” in its range query. This is exactly what the phantom read anomaly means, and it is what Serializable (next) is designed to close off.

(d) SERIALIZABLE

The strictest level. Transactions are guaranteed to execute as if they ran one at a time, completely serially, with zero anomalies — no dirty reads, no non-repeatable reads, no phantom reads. Usually implemented via full locking or by detecting conflicts and forcing one transaction to abort/retry.

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

Step	Transaction T1	Transaction T2
1	BEGIN TRANSACTION;	
2	SELECT SUM(balance) FROM Account WHERE branch = 'Delhi'; -- reads 50000	
3		BEGIN TRANSACTION; INSERT INTO Account (acc_id, branch, balance) VALUES ('Z9', 'Delhi', 3000);
4		-- tries to COMMIT... -- BLOCKED / forced to abort: -- would create a phantom row -- conflicting with T1's range
5	SELECT SUM(balance) FROM Account WHERE branch = 'Delhi'; -- STILL reads 50000 (no phantom)	
6	COMMIT;	-- can now safely retry & insert

The database guarantees T1's range query is never disturbed by a new matching row appearing mid-transaction. The trade-off is heavy locking or frequent “serialization failure, please retry” errors under high concurrency, so throughput drops compared to weaker levels.

Summary Table: All Four Levels Side by Side

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Relative Speed
Read Uncommitted	Possible	Possible	Possible	Fastest
Read Committed	Prevented	Possible	Possible	Fast
Repeatable Read	Prevented	Prevented	Possible	Moderate
Serializable	Prevented	Prevented	Prevented	Slowest

When to Use Which Isolation Level

Choosing an isolation level is really answering a small set of questions about what your transaction actually does. Walk through these in order:

- Does the transaction only need to read/write once, with no repeated reads of the same data? → Read Committed is almost always enough.
- Does the transaction read the same row(s) more than once and assume the value won't change in between? → You need at least Repeatable Read.
- Does the transaction run a range query (e.g., COUNT/SUM with a WHERE clause) that must not be affected by rows inserted by someone else mid-transaction? → You need Serializable.
- Is this dashboard/analytics-style data where being slightly stale or approximate is genuinely fine, and raw speed matters most? → Read Uncommitted can be considered — but only there.

In practice, most of an application runs fine on the engine's default (Read Committed in PostgreSQL/Oracle/SQL Server, Repeatable Read in MySQL's InnoDB). You then selectively raise the isolation level for the few specific transactions identified by the questions above — rather than changing it globally.

Read Uncommitted — *fastest, least safe*

Default in: Not the default in any major RDBMS; usually opt-in via a hint (e.g., SQL Server's NOLOCK).

Use when:

- You need a rough, approximate number and can tolerate it being briefly wrong.
- The query is for internal monitoring/observability, not a business decision.
- You want to avoid a read blocking on a long-running write, at any accuracy cost.

Real examples:

- An internal ops dashboard showing an approximate 'orders in progress' counter.
- A monitoring query estimating queue backlog size for an alert, not a payout.

Avoid when: Anything touching money, inventory counts used for selling, or any output a user or downstream system will act on.

Read Committed — *the safe everyday default*

Default in: PostgreSQL, Oracle, SQL Server, and most managed cloud OLTP databases.

Use when:

- Standard CRUD: create/read/update/delete operations that touch a row once per statement.
- Web and mobile app backends where each request is its own short transaction.
- General reporting where 'accurate as of right now' is good enough.

Real examples:

- Loading a product page, updating a user's profile, an admin panel showing current order status.
- A checkout flow that reads stock once, then updates it once, in the same short transaction.

Avoid when: A transaction that must re-read the same value later and treat it as unchanged (use Repeatable Read instead).

Repeatable Read — *consistent snapshot within a transaction*

Default in: MySQL / InnoDB (its standard default isolation level).

Use when:

- A transaction reads the same row(s) two or more times and later logic depends on that value not having moved.
- You are computing something in multiple steps from the same source data (e.g., validate, then act).
- You want protection from lost updates without paying for full Serializable locking.

Real examples:

- Generating a financial statement that re-reads the same account balance at several points and must show one snapshot.
- A multi-step order-validation flow that checks stock, then checks a price rule, then commits — using the same read.

Avoid when: High-throughput, high-contention tables where holding a consistent snapshot per transaction would create excess locking/versioning overhead.

Serializable — *strictest, safest, slowest*

Default in: SQLite (whole-database locking by default); an explicit opt-in everywhere else.

Use when:

- The transaction runs a range query (COUNT/SUM/aggregate with a WHERE clause) that must be immune to new matching rows appearing mid-transaction.
- Correctness is legally or financially non-negotiable — there is no acceptable margin for a race condition.
- You are enforcing a hard global invariant across many rows (a cap, a uniqueness rule spanning a range).

Real examples:

- A bank fund transfer, or the final step of confirming a seat/ticket booking.
- Enforcing 'total allocated budget across all cost-center rows must never exceed \$1,000,000.'

Avoid when: Applying it application-wide by default — use it narrowly, for the specific step that needs it, not the whole request lifecycle.

Practical guideline

Most production systems use Read Committed by default for everyday operations, and selectively escalate to Repeatable Read or Serializable only for the specific critical transactions that truly need it (e.g., wrap just the seat-booking confirmation or fund-transfer step in a stricter isolation level) — because applying the strictest level everywhere would unnecessarily slow the whole system down.

Practical Scenario

Scenario: Warehouse inventory clerks

Two clerks simultaneously view a stock count of 1 unit for the same SKU and both try to sell it in-store. Row-level locking (an isolation mechanism) ensures the second clerk's transaction either waits and then sees stock = 0, or is told the sale failed — preventing overselling a physical item that only exists once.

Failure scenario without isolation

An e-commerce flash sale with 10 units left in stock receives 500 simultaneous checkout attempts. Without proper isolation, dozens of transactions could all read stock = 10 before any commits, causing the system to accept far more orders than inventory actually allows.

6.2.4 Durability — “Once Committed, Always Committed”

Durability guarantees that once a transaction is committed, its changes are permanent and will survive any subsequent system failure — power loss, crash, or restart.

Example 1: Order Confirmation Before Crash

```
BEGIN TRANSACTION;  
  INSERT INTO Orders VALUES (1001, 'Laptop', 55000);  
COMMIT;  -- server responds: 'Order placed successfully'  
-- Power failure occurs 1 second later  
--  
-- After restart:  
SELECT * FROM Orders WHERE order_id = 1001;  
-- Result: row IS present. The commit was durable.
```

The DBMS achieves this by writing the transaction log entry to non-volatile disk storage before acknowledging the commit to the application. Even if the actual data file update hadn't been flushed to disk yet, the log itself is enough to “redo” the change on restart.

Example 2: Bank ATM Withdrawal

You withdraw ₹2000 from an ATM, and it prints a receipt confirming the transaction. Even if the bank's core server crashes moments later, the reduced balance and the withdrawal record must still be present when the server comes back online — the customer already has the physical cash in hand, so the record cannot simply disappear.

Example 3: Distributed Databases

In distributed SQL/NoSQL systems (e.g., CockroachDB, Amazon Aurora, Google Spanner), durability often means the write must be replicated and acknowledged by a quorum of nodes (say, 2 out of 3 replicas) before the commit is confirmed to the client — ensuring that even if one entire server/data center is destroyed, the committed data survives on the others.

How Durability is Implemented

- Write-Ahead Logging (WAL) flushed to disk before commit acknowledgment.
- Checkpointing: periodically flushing in-memory data pages to disk so recovery from the log is faster.
- Replication: copying committed data to multiple nodes/data centers so a single hardware failure cannot cause data loss.

Practical Scenario

Scenario: Hospital patient records system

A doctor updates a patient's medication record and the system confirms the save. If the hospital's database server loses power 2 seconds later, the updated medication record absolutely must still be there on reboot — a missing update to a medication record could directly endanger patient safety.

6.3 Serializability

A schedule is the order in which operations from multiple transactions are executed. A schedule is called serializable if its result is guaranteed to be the same as some serial (one-after-another, no overlap) execution of the same transactions. Serializability is the formal correctness goal that isolation levels like Serializable are designed to achieve.

6.3.1 Simple Example: Serial vs. Non-Serial Schedule

Two transactions, both on the same item X (starting value X = 100):

```
T1: Read(X); X = X + 50; Write(X);    -- adds 50
T2: Read(X); X = X * 2;  Write(X);    -- doubles X
```

Schedule Type	Order of Operations	Final Value of X
Serial (T1 then T2)	R1(X)=100, X=150, W1(X); R2(X)=150, X=300, W2(X)	300
Serial (T2 then T1)	R2(X)=100, X=200, W2(X); R1(X)=200, X=250, W1(X)	250
Interleaved (non-serial)	R1(X)=100; R2(X)=100; W1(X)=150; W2(X)=200	200 (wrong — matches neither serial order)

The interleaved schedule produces 200 — a result that does not match either valid serial order (300 or 250). It is therefore not serializable, and would be rejected/prevented by the concurrency control mechanism.

6.3.2 Conflict Serializability

Two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a Write. A schedule is conflict serializable if we can reorder its non-conflicting operations to turn it into a serial schedule.

Simple example schedule S on the same item X:

```
S:  R1(X)  R2(X)  W1(X)  W2(X)
```

Here $W1(X)$ and $W2(X)$ conflict (both write X), and $R2(X)/W1(X)$ also conflict (one reads, one writes the same item). Using a precedence graph, we draw an edge $T1 \rightarrow T2$ whenever an operation of $T1$ must come before a conflicting operation of $T2$:

- $R1(X)$ before $W2(X) \rightarrow$ edge $T1 \rightarrow T2$
- $W1(X)$ before $W2(X) \rightarrow$ edge $T1 \rightarrow T2$
- $R2(X)$ before $W1(X) \rightarrow$ edge $T2 \rightarrow T1$

This produces both $T1 \rightarrow T2$ and $T2 \rightarrow T1$, which is a cycle. A cycle in the precedence graph means the schedule is NOT conflict serializable — there is no way to reorder it into an equivalent serial schedule. If the graph has no cycles, the schedule is conflict serializable, and a valid serial order can be read directly off the graph.

Core takeaway

Build a precedence graph: one node per transaction, one edge per conflicting operation pair (in the order they appear in the schedule). Acyclic graph = conflict serializable (safe). Cyclic graph = not serializable (concurrency control must prevent this schedule from happening).

6.3.3 View Serializability (Brief)

View serializability is a broader (less strict) notion: a schedule is view serializable if each transaction reads the same values and produces the same final writes as in some serial schedule, even if the exact conflicting operation order differs. Every conflict serializable schedule is view serializable, but not every view serializable schedule is conflict serializable. In practice, database engines test for conflict serializability because it is easier to check efficiently (via the precedence graph), while checking view serializability in general is computationally much harder.

6.4 Concurrency Control

Concurrency control is the set of techniques a DBMS uses to make sure that when multiple transactions run at the same time, the result is correct — equivalent to running them one at a time — while still allowing as much parallelism as possible for performance. It is the actual machinery that enforces the Isolation property and serializability discussed above.

6.4.1 Need for Concurrency Control

If transactions were simply allowed to run concurrently with no coordination, several well-known problems occur. Here is the simplest possible example of each, using a single shared counter.

(a) Lost Update Problem

A shared counter starts at 5. Two transactions each try to increment it by 1:

Step	Transaction T1	Transaction T2
1	Read Counter → 5	Read Counter → 5
2	Counter = 5 + 1 = 6	
3	Write Counter = 6	
4		Counter = 5 + 1 = 6 (T2 used its stale copy)
5		Write Counter = 6

Final Counter = 6, but two increments happened — the correct expected value was 7.

One update is silently lost because T2 never saw T1's change before computing its own.

(b) Dirty Read Problem

Covered earlier under Isolation (6.2.3) — a transaction reads a value written by another transaction that later rolls back, so it acted on data that never really existed.

(c) Unrepeatable Read Problem

Covered earlier under Isolation (6.2.3) — the same query inside one transaction returns two different values because another transaction changed and committed data in between.

(d) Incorrect Summary (Aggregate) Problem

A transaction T1 is summing balances across 3 accounts (A=100, B=100, C=100, expected total = 300). While T1 is midway through reading them, T2 transfers 50 from C to A and commits. T1 might read A's new value (150) but C's old value (100, before T2's debit hadn't happened when T1 read it) — giving an incorrect total like 350, which never existed at any real point in time.

Why This Matters

Need for concurrency control

Without concurrency control, a system that looks perfectly fine in single-user testing can silently corrupt data the moment real traffic with multiple simultaneous users arrives — lost updates, wrong balances, and inconsistent reports. Concurrency control mechanisms (locks, timestamps, or multiversioning) are what let a database safely serve

thousands of users at once without sacrificing correctness.

6.4.2 Lock-Based Protocols

The most common way to implement concurrency control is locking: a transaction must acquire a lock on a data item before operating on it, and other transactions needing a conflicting lock on that same item must wait.

Types of Locks

Lock Type	Also Called	Allows Others To...	Used For
Shared Lock	S-lock, Read lock	Also take a Shared lock (multiple readers OK)	Reading a data item
Exclusive Lock	X-lock, Write lock	Nothing — no other lock allowed until released	Writing/updating a data item

Simple compatibility rule: many transactions can hold a Shared lock on the same item at once, but an Exclusive lock is always alone — no other S or X lock can coexist with it.

Simple Example: Locking in Action

Step	T1	T2	T3
1	LOCK-S(X) Read(X)		
2		LOCK-S(X) Read(X) (S + S is OK)	
3			LOCK-X(X) must WAIT for T1 and T2 to unlock
4	UNLOCK(X)		
5		UNLOCK(X)	
6			LOCK-X(X) granted Write(X) UNLOCK(X)

Two-Phase Locking (2PL)

Simply using locks is not enough to guarantee serializability on its own — transactions must also follow a disciplined protocol for when they acquire and release locks. The Two-Phase Locking protocol splits every transaction into two phases:

- Growing Phase: the transaction may acquire (lock) new data items, but may not release any lock.
- Shrinking Phase: the transaction may release locks, but may not acquire any new lock.

Once a transaction releases its first lock, it has entered the shrinking phase and can never acquire another lock.

```
T1: LOCK-S(A)  LOCK-X(B)  LOCK-S(C) |  
    UNLOCK(A)  UNLOCK(C)  UNLOCK(B)  
          <----- Growing -----> | <-----  
          ---- Shrinking ----->
```

Core guarantee of 2PL

Any schedule in which every transaction follows the Two-Phase Locking rule is guaranteed to be conflict serializable. This is the theorem that makes 2PL the standard foundation for lock-based concurrency control in real databases.

Strict Two-Phase Locking

Basic 2PL can still allow one transaction to read a value written by another transaction that has released its lock but not yet committed — risking a dirty read if that transaction later aborts. Strict 2PL fixes this with one extra rule: all Exclusive locks are held until the transaction actually commits or rolls back (not just until the shrinking phase begins). This is what most commercial databases actually implement, because it also makes recovery simpler.

A Note on Deadlocks

Locking introduces a new risk: deadlock — e.g., T1 holds a lock on A and waits for B, while T2 holds a lock on B and waits for A; neither can proceed. Databases handle this with deadlock detection (periodically checking a wait-for graph for cycles and aborting one transaction) or deadlock prevention (e.g., ordering lock requests, or aborting a transaction that would have to wait on a lock held by an “older” or “younger” transaction, depending on the scheme used).

6.5 Recovery

Recovery is the set of techniques a DBMS uses to bring the database back to a correct, consistent state after a failure — this is what actually delivers the Durability and Atomicity guarantees in practice.

6.5.1 Failure Classification

Failure Type	Description	Simple Example
Transaction Failure — Logical Error	The transaction itself can no longer continue due to a bad internal condition (e.g., invalid input, constraint violation, code bug).	A withdrawal transaction tries to debit more money than the account balance allows, and the CHECK constraint rejects it.
Transaction Failure — System Error	The DBMS itself puts a transaction into an undesirable state and terminates it (e.g., deadlock detected, resource limit hit).	Two transactions deadlock waiting on each other's locks; the DBMS aborts one of them automatically.
System Crash	A hardware or software failure (power loss, OS crash, bug) causes the whole database server to stop, wiping the contents of volatile memory (RAM). Data already written to non-volatile disk storage is assumed safe.	The database server loses power in the middle of processing several open transactions.
Disk / Media Failure	The physical storage device itself is damaged or corrupted, and some or all of the data on disk becomes unreadable.	A hard drive head crash physically destroys the sectors storing part of a database file.

6.5.2 Shadow Paging (Recovery Technique)

Shadow paging is an alternative to log-based recovery. Instead of logging every change, it keeps two page tables:

- Shadow Page Table — points to the original, unmodified data pages on disk. This is never changed while the transaction runs.
- Current Page Table — used by the running transaction; whenever it wants to modify a page, a copy of that page is made first, and the Current Page Table is updated to point to the new copy instead of the original.

Simple walkthrough:

Before transaction starts:

Shadow Page Table ---> Page 1, Page 2, Page 3 (on disk)

Current Page Table ---> Page 1, Page 2, Page 3 (same as shadow, initially)

Transaction modifies Page 2:

1. Make a NEW copy: Page 2' (changes are written here, original Page 2 untouched)

2. Current Page Table ---> Page 1, Page 2', Page 3

3. Shadow Page Table ---> Page 1, Page 2, Page 3 (unchanged, still the old version)

If the transaction COMMITS:

Shadow Page Table is simply replaced by the Current Page Table (one pointer swap)

-> Page 2' is now the permanent, official version

If the transaction CRASHES or ABORTS before commit:

Just discard the Current Page Table and Page 2'

Shadow Page Table (still pointing at the original Page 2) is untouched and remains correct

Because the shadow copy is never modified until the final commit is a single atomic pointer switch, there is no need to undo anything on crash recovery — the old, consistent version is always safely available. This makes recovery extremely fast. The main drawbacks are that it complicates supporting concurrent transactions, and it causes data fragmentation and extra garbage-collection overhead as old, orphaned pages accumulate on disk.

Shadow Paging vs. Log-Based Recovery

Log-based recovery (Write-Ahead Logging) is used by almost all modern production databases because it handles concurrent transactions and partial updates more efficiently. Shadow paging is conceptually simpler and avoids undo/redo logic entirely, but is less practical at scale — it is mostly taught as a foundational recovery concept.

6.6 Putting It All Together: Full Bank Transfer Walkthrough

Consider the complete transfer of ₹5000 from Account A to Account B:

```
BEGIN TRANSACTION;
  UPDATE Account SET balance = balance - 5000
    WHERE acc_id = 'A';    -- must not violate balance >= 0

  UPDATE Account SET balance = balance + 5000
    WHERE acc_id = 'B';
COMMIT;
```

Property	How it applies here
Atomicity	Both UPDATES succeed together, or if the system crashes mid-way, both are rolled back. Money is never partially transferred.
Consistency	The CHECK constraint (balance >= 0) and the invariant 'total money in the system is unchanged' are preserved before and after the transaction.
Isolation	If another transaction simultaneously queries Account A's balance, it sees either the pre-transfer or post-transfer value — never a half-updated intermediate state.
Durability	Once the application

	shows 'Transfer Successful' to the user, the change survives even an immediate server crash.
--	---

6.7 Why ACID Matters — Real-World Stakes

Property	If Violated...
Atomicity	Money is debited from one account but never credited to another (partial failure), or a customer is charged without receiving their reserved item.
Consistency	The database ends up holding invalid data — negative stock counts, mismatched order totals, orphaned foreign key references.
Isolation	Two customers book the same seat/item (lost updates), or one transaction reads a value that another transaction later rolls back (dirty reads).
Durability	A confirmed transaction (payment, booking, medical record update) silently disappears after a crash, even though the user was told it succeeded.

6.7.1 Where ACID is Critical

- Banking and financial systems — fund transfers, ledger entries, interest calculations
- E-commerce — order placement, payment processing, inventory deduction
- Airline/hotel/ticket booking systems — seat and room reservation
- Healthcare records — medication history, lab results, patient charts
- Inventory and supply chain management — stock levels across warehouses

6.7.2 The Trade-off: ACID vs. BASE

Traditional relational databases (PostgreSQL, MySQL/InnoDB, Oracle, SQL Server) are built around strict ACID guarantees. Many distributed NoSQL databases (Cassandra, DynamoDB, and MongoDB in certain configurations) intentionally relax some ACID properties — particularly strict isolation and immediate consistency — in favor of higher availability and partition tolerance, per the CAP theorem. This relaxed model is often called BASE: Basically Available, Soft state, Eventually consistent.

Use Case	Recommended Model	Reasoning
Bank balance transfer	ACID	Cannot tolerate even momentary incorrect balances.
Social media 'like' counter	BASE	A few seconds of lag or a temporarily approximate count is acceptable.
Airline seat reservation	ACID	Double-booking a physical seat is unacceptable.
Product view/analytics counter	BASE	High write volume matters more than perfect real-time accuracy.

6.8 Summary

- Atomicity — all operations in a transaction succeed, or none do.
- Consistency — every transaction moves the database from one valid state to another, respecting all constraints.
- Isolation — concurrent transactions behave as if executed one at a time, preventing race conditions like dirty reads and lost updates.
- Durability — once committed, a transaction's effects survive crashes and power failures permanently.

Together, these four properties are what make relational databases dependable enough to run banks, hospitals, airlines, and online marketplaces — systems where correctness is non-negotiable.